

CONCEPTUAL DESIGN FOR AN END-TO-END DATA PIPELINE

Beejan Technologies | Telecommunications | Customer Experience & Operations

Data Sources: Social Media, Call Centre Interaction Records, SMS, Website/App Forms

Design perspective: Most architectural decisions are informed by my exposure to the telecommunications environment, particularly the relationship between network incidents, customer experience and operational data. I also use relevant MTN use cases and public technical references where the scenario does not provide specific requirements.

1. The Business Scenario

Beejan Technologies receives thousands of customer complaints every day concerning poor network service, incorrect billing and poor customer service. Complaints arrive through social media, call-centre channels, SMS and website/application forms. The data is stored separately and in different formats, requiring manual spreadsheet consolidation. This creates reporting delays, fragmented information and limited visibility across teams. The proposed pipeline brings the sources together, preserves the original information, processes it consistently and supports both **near-real-time operational visibility and historical analysis**.

2. Design Requirements

The assignment specifies seven design considerations: **data sourcing, ingestion, processing and transformation, storage, data serving, orchestration and monitoring, and DataOps**. The decisions below are based on source characteristics, the business use case and relevant telecommunications experience.

3. Proposed Solution Architecture

3.1 Data Source Analysis

The pipeline combines all four sources, but I would not treat them identically because their **formats, delivery patterns and operational value** differ. Social-media content can include unstructured text, images and videos [1]. I assume significant complaint-volume changes should become visible within **1–2 minutes**. Publicly reported MTN outages, where customers simultaneously reported problems with calls, SMS and data services, influenced this assumption [2]. Call-centre data may arrive as CSV/text and include interaction records, CDR/EDR-type records, logs and transcripts [4]. I assume these records can build up and be processed in **batch** because the complaint-analysis use case does not require the same low latency as social-media monitoring. SMS is treated as message-based data, potentially JSON/XML, with an assumed **30-minute freshness target**. Website/app complaints are assumed to require approximately **10-minute freshness** because they are direct customer-service requests that should reach the responsible team promptly, but do not require second-by-second processing.

3.2 Ingestion Strategy

The ingestion layer uses a **hybrid approach**. I would use **incremental ingestion** for ongoing data, with a full load only for an initial historical backfill. For social media, the underlying content is unstructured, but the platform API can deliver posts and metadata as semi-structured JSON [1][2]. I would favour **push-based streaming**; X's Filtered Stream is an example of receiving relevant posts as they are published [3]. Website/app submissions can use APIs or webhooks. Where push delivery is unavailable, the pipeline can **pull incrementally**, retrieving only new records. SMS can use a message queue to buffer incoming messages, while call-centre records can be pulled on a schedule for batch processing. Because complaint volumes may rise sharply during incidents, the ingestion layer should support **buffering, fault tolerance and horizontal scaling** to absorb, for instance, a 10× spike without data loss or service degradation.

3.3 Processing & Transformation

The processing layer will use a **Lambda architecture** because the pipeline has both streaming and batch data. Social-media complaints require near-real-time processing, while call-centre records can be processed in batches [6]. I would first load incoming data into the **raw data-lake layer** before transforming it. This follows **ELT (Extract, Load, Transform)** and preserves the original data for reprocessing if errors or rule changes occur. Data will be cleaned, validated and standardized, including handling missing/invalid values, standardizing timestamps and text, identifying duplicates, and normalizing text while preserving meaningful elements such as emojis for sentiment analysis. Complaints will be classified into **network, billing and customer service**; NLP can support classification and sentiment analysis can indicate dissatisfaction. Customer/location information can be added where available, while severity and **SLA** requirements can help determine priority. The result is curated data for analytical storage, keeping OLAP workloads separate from operational OLTP systems.

3.4 Storage Architecture

The storage strategy uses a **raw and curated data lake alongside a structured data warehouse**. The raw layer preserves original data, lineage and the ability to reprocess it. Because sources have different and changing formats, the data lake will use **schema-on-read**, avoiding one rigid schema at ingestion. Operational systems commonly use **row-oriented storage** for OLTP workloads such as frequent inserts and updates. In the lake, curated data can be stored in **Parquet (columnar)** for efficient analytical processing, then loaded into a **structured data warehouse** for OLAP queries such as complaint trends by location, category, channel and time. The trade-off is additional storage/management overhead in exchange for raw-data preservation, flexibility, reduced impact from source-schema changes and efficient analytics.

3.5 Data Serving

The serving layer provides different access paths for downstream users. **Dashboards** give operations and customer-service teams visibility into complaint volumes, emerging issues, locations and priorities. Analysts can use **SQL/OLAP queries** for historical trends, cross-channel analysis and performance reporting. **Alerts** can flag complaint spikes, high-priority complaints and potential SLA breaches. APIs can support enterprise integration, while scheduled reports can support management and business users.

3.6 Orchestration & Monitoring

The pipeline runs according to the freshness requirements defined for each source: social-media streaming runs continuously, while SMS, website/app and call-centre workloads run at their respective intervals. An orchestration layer manages workflows, dependencies and retries. Failures are detected through **health checks, job-status monitoring and data-freshness checks**. Alerts are triggered when jobs fail, data stops arriving or a freshness target is missed.

3.7 DataOps

DataOps will focus on making the pipeline reliable and maintainable in production. Pipeline changes will be **version-controlled and tested** before deployment, with a **rollback mechanism** to return to the last known-good version if a change causes unexpected results. This is informed by my telecom exposure, where rollback procedures are prepared for network changes that produce unintended effects. The pipeline should also support fault recovery, horizontal scaling, access control and data-quality checks.

4. Key Assumptions & Thought Process

- **Social media:** Significant complaint-volume changes should be visible within approximately 1–2 minutes, influenced by telecom outage scenarios and the value of social media as an operational signal [2].
- **Website/App:** A 10-minute freshness target is assumed because these are direct customer-service requests that should reach the responsible team promptly.
- **SMS:** A 30-minute freshness target is assumed as a balance between responsiveness and processing overhead.
- **Call centre:** Records can be processed in batch because the main use case is complaint analysis and reporting rather than immediate incident detection.
- **Volume:** The pipeline should handle a temporary 10× increase during a major service disruption. This is a stress assumption, not a claim about Beejan's actual traffic.
- **Ingestion:** Ongoing movement is assumed to be incremental; full extraction is mainly for an initial historical load.
- **Storage:** Raw data should be preserved to support reprocessing, lineage and auditability.

5. Challenges & Unknowns

The scenario does not specify the exact social-media platforms, API limits, source volumes, retention requirements, cross-channel customer-identification method or existing SLAs. These would need to be confirmed before implementation. The structure and delivery method of call-centre CDR/EDR and transcript data would also need to be assessed because they may differ between source systems. Privacy and access requirements for customer information would also need to be defined before production deployment.

Design Summary

Sources → **Ingestion** → **Raw Data Lake** → **Processing** → **Curated Data** → **Data Warehouse** → **Serving**, with orchestration, monitoring and DataOps operating across the pipeline. The architecture deliberately separates fast operational signals from batch analytical workloads while preserving raw data for recovery and reprocessing.

References

- [1] Data Knowledge Hub for Researching Online Discourse — *Social Media Data Types: An Overview*.
- [2] PUNCH Online — *MTN Customers Lament Communication Blackout* (8 February 2025), documenting widespread customer complaints during an MTN outage.
- [3] X Developer Platform — *Filtered Stream API Quickstart*.
- [4] Microsoft Learn — *CDR/EDR Analytics Architecture with Real-Time Intelligence*.
- [5] Jaeyk — *Computational Thinking for Social Scientists, Chapter 7: Semi-structured Data*.
- [6] Lewis Sawe — *Designing Efficient Complaint Data Pipeline for Beejan Technologies*.